

Some Large-Scale Matrix Computation Problems

Vladislav Meshkov, MSc.

Moscow Institute of Physics and Technology

December 18, 2025

Table of contents

Task/Model Motivation

Formal Problem Statement

Algorithmic Approach

Applications

Monte Carlo Acceleration

Numerical Experiments

Conclusions

Task/Model Motivation

- ▶ Large-scale problems require efficient methods
Computing quantities like $\text{tr}(A^{-1})$ and $\det(A)$ for matrices of order millions arises in physics (lattice QCD), fractals, and circuit simulation—traditional $O(n^3)$ methods are impractical
- ▶ Matrix-vector products are the computational bottleneck
For sparse or structured matrices, forming Ax is much cheaper than full matrix operations—need algorithms that only access A through matrix-vector multiplications
- ▶ Existing quadrature-based approaches need practical refinement
Golub-Meurant methods provide theoretical foundation using Lanczos and Gaussian quadrature, but lack practical implementation details for non-SPD matrices and confidence bounds

Formal Problem Statement

Given:

- ▶ An $n \times n$ real matrix A (possibly non-symmetric)
- ▶ Vectors $u, v \in \mathbb{R}^n$
- ▶ Smooth function f defined on the spectrum of A

Central Problem:

$$L \leq u^T f(A) v \leq U$$

Key Applications:

- ▶ Entries of matrix inverse: $u = v = e_i, f(\lambda) = 1/\lambda \Rightarrow (A^{-1})_{ii}$
- ▶ Trace of inverse: $\sum_{i=1}^n (A^{-1})_{ii} = \text{tr}(A^{-1})$
- ▶ Determinant: $\det(A) = \exp(\text{tr}(\ln(A)))$ via $f(\lambda) = \ln \lambda$

Goal: Develop algorithms using only **matrix-vector products** Ax

$\Rightarrow$ Memory: $O(3n + \kappa)$, Cost: $O(jv)$ where j = iterations, v = matvec cost

Core Idea: Quadrature + Lanczos

Step 1: Transform to Riemann-Stieltjes Integral

For symmetric $A = Q^T \Lambda Q$:

$$u^T f(A) v = \sum_{i=1}^n f(\lambda_i) \hat{u}_i \hat{v}_i = \int_a^b f(\lambda) d\mu(\lambda)$$

where $\mu(\lambda)$ is piecewise constant measure from eigenvalue distribution

Step 2: Approximate via Gaussian Quadrature

General formula:

$$I[f] = \sum_{j=1}^N \omega_j f(\theta_j) + \sum_{k=1}^M \nu_k f(\tau_k)$$

- ▶ **Gauss rule** ($M = 0$): nodes $\{\theta_j\}$ are zeros of orthogonal polynomial $p_j(\lambda)$
- ▶ **Gauss-Radau** ($M = 1$): prescribe one endpoint a or b
- ▶ **Gauss-Lobatto** ($M = 2$): prescribe both endpoints a and b

Key: Nodes and weights computed via **Lanczos tridiagonalization** 

Lanczos Procedure and Orthogonal Polynomials

Three-Term Recurrence for Orthonormal Polynomials:

$$\gamma_j p_j(\lambda) = (\lambda - \alpha_j) p_{j-1}(\lambda) - \gamma_{j-1} p_{j-2}(\lambda)$$

with $p_{-1}(\lambda) = 0$, $p_0(\lambda) = 1$

Matrix Form: $A p(\lambda) = T_j p(\lambda) + \gamma_j p_j(\lambda) e_j$

Tridiagonal matrix:

$$T_j = \begin{bmatrix} \alpha_1 & \gamma_1 & & \\ \gamma_1 & \alpha_2 & \gamma_2 & \\ & \ddots & \ddots & \gamma_{j-1} \\ & & \gamma_{j-1} & \alpha_j \end{bmatrix}$$

Quadrature Construction:

- ▶ Nodes $\{\theta_j\}$: eigenvalues of T_j (or adjusted $\tilde{T}_j$ for Radau/Lobatto)
- ▶ Weights $\{\omega_j\}$: squares of first elements of normalized eigenvectors
- ▶ Approximation: $I_j = \sum_{k=1}^j \omega_k^2 f(\theta_k)$

Algorithm 1: Basic Quadrature-Lanczos Method

Input: SPD matrix $A \in \mathbb{R}^{n \times n}$, $\lambda(A) \subseteq [a, b]$, vector u , function f

Initialization: $x_0 = u/\|u\|_2$, $x_{-1} = 0$, $\gamma_0 = 0$

For $j = 1, 2, \dots$ **until convergence:**

1. $\alpha_j = x_{j-1}^T A x_{j-1}$
2. $r_j = A x_{j-1} - \alpha_j x_{j-1} - \gamma_{j-1} x_{j-2}$
3. $\gamma_j = \|r_j\|_2$
4. Adjust T_j to have prescribed eigenvalues (for Radau/Lobatto) $\rightarrow \tilde{T}_j$
5. Compute eigenvalues $\{\theta_k\}$ and first eigenvector elements $\{\omega_k\}$ of $\tilde{T}_j$
6. $l_j = \|u\|_2^2 \sum_{k=1}^j \omega_k^2 f(\theta_k)$
7. Check convergence: $|l_j - l_{j-1}| < \delta |l_j|$
8. $x_j = r_j / \gamma_j$

Output: Lower bound L and/or upper bound U of $u^T f(A) u$

Complexity: Memory $O(3n)$, Cost $O(jv)$ where v is cost of Ax

Application 1: Bounding Matrix Inverse Entries

Diagonal Elements $(A^{-1})_{ii}$:

Set $u = v = e_i$ and $f(\lambda) = 1/\lambda$, then $u^T f(A) u = (A^{-1})_{ii}$

- ▶ **Gauss rule:** lower bound L (since $f^{(2n)}(\xi) > 0$)
- ▶ **Gauss-Radau:** both L and U (one endpoint prescribed)
- ▶ **Gauss-Lobatto:** upper bound U (both endpoints prescribed)

Off-Diagonal Elements $(A^{-1})_{ij}$:

Use polarization identity:

$$(A^{-1})_{ij} = \frac{1}{4}(y^T A^{-1} y - z^T A^{-1} z)$$

where $y = e_i + e_j$ and $z = e_i - e_j$

Efficiency Trick: Avoid computing eigendecomposition explicitly

For $f(\lambda) = 1/\lambda$: compute $(T_j^{-1})_{11}$ recursively [Golub-Meurant]

Application 2: Non-SPD Matrices via Normal Equations

Problem: What if A is not symmetric positive definite?

Key Observation: For nonsingular A , matrix $A^T A$ is SPD and

$$A^{-1} = (A^T A)^{-1} A^T$$

Strategy for $(A^{-1})_{ij}$:

$$e_i^T A^{-1} e_j = e_i^T (A^T A)^{-1} A^T e_j = e_i^T (A^T A)^{-1} v$$

where $v = A^T e_j$. Apply Algorithm 1 with $u = e_i$, v , and matrix $A^T A$

Strategy for $tr(A^{-1})$:

$$tr(A^{-1}) = \frac{1}{2} tr(A^{-1} + A^{-T}) = \frac{1}{2} tr((A^T A)^{-1} A^T + A(A^T A)^{-1})$$

Each diagonal term: $2e_i^T (A^T A)^{-1} A^T e_i$ with $v = A^T e_i$

Note: Never form $A^T A$ explicitly—compute $A^T A x$ in two matvecs!

Application 3: Determinant via Trace of Logarithm

Mathematical Identity: For SPD matrix A

$$\det(A) = \exp(\text{tr}(\ln(A)))$$

Algorithm:

1. Set $f(\lambda) = \ln \lambda$
2. Apply Algorithm 1 to bound $(\ln(A))_{ii}$ for each $i = 1, \dots, n$
3. Sum bounds: $\sum_{i=1}^n L_i \leq \text{tr}(\ln(A)) \leq \sum_{i=1}^n U_i$
4. Exponentiate: $\exp(\sum L_i) \leq \det(A) \leq \exp(\sum U_i)$

Derivative Signs:

- ▶ $f^{(2n+1)}(\lambda) = (-1)^n (2n)! \lambda^{-(2n+1)} > 0$
- ▶ $f^{(2n)}(\lambda) = (-1)^{n+1} (2n-1)! \lambda^{-2n} < 0$

⇒ Gauss rule gives **upper bound**, Radau gives both bounds

Bottleneck: Running Algorithm 1 n times costs $O(njv) \rightarrow$ need Monte Carlo!

Monte Carlo Approach for Trace Estimation

Key Theorem (Hutchinson's Estimator):

Let $H \in \mathbb{R}^{n \times n}$ be symmetric, $z \in \{-1, +1\}^n$ with each entry uniform. Then:

$$\mathbb{E}[z^T H z] = \text{tr}(H), \quad \text{var}(z^T H z) = 2 \sum_{i \neq j} H_{ij}^2$$

Practical Algorithm:

1. Generate m random vectors $z_1, \dots, z_m$ (each entry ± 1 with prob. 0.5)
2. For each z_i , apply Algorithm 1 to obtain bounds:

$$L_i \leq z_i^T f(A) z_i \leq U_i$$

3. Estimate trace via average:

$$\frac{1}{m} \sum_{i=1}^m L_i \leq \text{tr}(f(A)) \leq \frac{1}{m} \sum_{i=1}^m U_i$$

Advantage: Reduce cost from $O(njv)$ to $O(mjv)$ where $m \ll n$.

Confidence Bounds via Hoeffding's Inequality

Hoeffding's Inequality:

Let $W_1, \dots, W_m$ be independent, zero-mean, bounded: $a_i \leq W_i \leq b_i$. Then:

$$P\left(\left|\sum_{i=1}^m W_i\right| \geq \eta\right) \leq 2 \exp\left(-\frac{2\eta^2}{\sum_{i=1}^m (b_i - a_i)^2}\right)$$

Application to Trace Estimation:

Let $W_i = z_i^T H z_i - \text{tr}(H)$, then $L_{\min} - \text{tr}(H) \leq W_i \leq U_{\max} - \text{tr}(H)$

Set $d = m(U_{\max} - L_{\min})^2$, then with probability $\geq 1 - 2 \exp(-2\eta^2/d)$:

$$\frac{1}{m} \sum_{i=1}^m L_i - \frac{\eta}{m} \leq \text{tr}(H) \leq \frac{1}{m} \sum_{i=1}^m U_i + \frac{\eta}{m}$$

Confidence Interval Width: For probability p :

$$\frac{\eta}{m} = \frac{1}{2m} (U_{\max} - L_{\min})^2 \ln\left(\frac{2}{1-p}\right)$$

Algorithm 2: Monte Carlo Trace Estimation

Input: SPD $A \in \mathbb{R}^{n \times n}$, function f , sample size m , probability p

For $j = 1, \dots, m$:

1. Generate random vector $z_j \in \{-1, +1\}^n$
2. Apply Algorithm 1 (Gauss-Radau) to get $L_j \leq z_j^T f(A) z_j \leq U_j$
3. Update: $L_{\min} = \min\{L_{\min}, L_j\}$, $U_{\max} = \max\{U_{\max}, U_j\}$
4. Compute running estimate: $I(j) = \frac{1}{2j} \sum_{i=1}^j (L_i + U_i)$
5. Compute confidence width:

$$\eta^2 = 0.5j(U_{\max} - L_{\min})^2 \ln(2/(1-p))$$

6. Update confidence bounds:

$$L_p(j) = \frac{1}{j} \sum_{i=1}^j L_i - \frac{\eta}{j}, \quad U_p(j) = \frac{1}{j} \sum_{i=1}^j U_i + \frac{\eta}{j}$$

Output: Estimator $I_m = I(m)$, confidence interval $[L_p(m), U_p(m)]$

Connection: Hessian Estimation via Trace Methods

Problem in ELBO Optimization:

Computing Hessian-dependent quantities for SGD analysis:

- ▶ Trace of Hessian: $\text{Tr}(H) = \sum_{i=1}^D H_{ii}$
- ▶ Squared Hessian: $\text{Tr}(H^2) = \sum_{i,j} H_{ij}^2$
- ▶ Log-determinant: $\log |I - \alpha H|$ for convergence analysis

Challenge: Direct computation of full Hessian $H \in \mathbb{R}^{D \times D}$ is prohibitive for high-dimensional models (deep neural networks with $D \sim 10^6$ – 10^9)

Solution via Lanczos Quadrature:

Use Algorithm 1 to **efficiently estimate**:

$$\text{Tr}(H) = \int_a^b d\mu(\lambda) \approx \sum_{k=1}^j \omega_k \quad (\text{Gauss quadrature})$$

where ω_k are weights from eigendecomposition of tridiagonal T_j (cost: only $O(jD)$ Hessian-vector products)

Hessian Estimation in SGD: From Theory to Practice

Entropy Change via Log-Determinant:

SGD step Jacobian: $J_t = I - \alpha H_t$, entropy change:

$$\Delta S_t = \log |I - \alpha H_t| = \sum_{i=1}^D \log(1 - \alpha \lambda_i)$$

Parabolic Lower Bound (from ELBO slides):

$$\log |I - \alpha H| \geq -\alpha \text{Tr}(H) - \alpha^2 \text{Tr}(H^2)$$

Efficient Trace Estimation (Hutchinson, 1989):

For random $z \sim N(0, I)$:

$$\mathbb{E}[z^T H z] = \text{Tr}(H), \quad \mathbb{E}[\|H z\|_2^2] = \text{Tr}(H^2)$$

Lanczos Integration:

Instead of computing exact Hessian, use Algorithm 1 with $f(\lambda) = \lambda$ and $f(\lambda) = \lambda^2$:

Experiment 1: Diagonal Inverse Entries

Test Matrix: Linear heat flow (finite difference discretization, $n = 900$)

i	Exact $(A^{-1})_{ii}$	Iter	Lower L_i	Upper U_i	$U_i - L_i$
1	5.7020150e-01	4	5.7020115e-01	5.7020202e-01	8.68e-07
22	5.7792259e-01	4	5.7792195e-01	5.7792349e-01	1.54e-06
32	5.8626306e-01	4	5.8626209e-01	5.8626430e-01	2.21e-06

Observations:

- ▶ Convergence in only **4 iterations**—extremely fast!
- ▶ Bounds are **very sharp**: gap $\sim 10^{-6}$ (relative error $< 10^{-5}$)
- ▶ Much sharper than variational bounds [Robinson-Wathen, 1992]:
 - ▶ Their bound for $(A^{-1})_{1,1}$: $[0.5676, 0.5750]$ (gap $\sim 10^{-2}$)
 - ▶ Our bound: $[0.57020115, 0.57020202]$ (gap $\sim 10^{-6}$)

Experiment 2: Vicsek Fractal Hamiltonian (VFH)

Test Matrix: Transverse vibration of Vicsek fractal (self-similar structure)

Results for $n = 3125$:

i	Iter	Lower L_i	Upper U_i	$U_i - L_i$
1	19	9.4801416e-01	9.4801776e-01	3.60e-06
100	20	1.1005253e+00	1.1005302e+00	4.90e-06
2000	19	1.1003685e+00	1.1003786e+00	1.01e-05
3125	16	6.4400234e-01	6.4401100e-01	8.66e-06

Key Features:

- ▶ Convergence in 13–20 iterations for $n = 3125$
- ▶ Sparse fractal structure $\Rightarrow$ cheap matrix-vector products
- ▶ Applications: vibrational modes of fractals, quantum physics

Experiment 3: Monte Carlo for $\text{tr}(A^{-1})$

Test Matrices: Various SPD matrices, $m = 50$ Monte Carlo samples

Matrix	n	Exact $\text{tr}(A^{-1})$	Estimated	Rel. Err.	95% Conf. Interval
Poisson	900	5.126e+02	5.020e+02	2.0%	(4.28e+02, 5.75e+02)
VFH	625	5.383e+02	5.366e+02	0.3%	(5.05e+02, 5.67e+02)
Wathen	481	2.681e+01	2.667e+01	0.5%	(2.58e+01, 2.75e+01)
Lehmer	200	2.000e+04	2.017e+04	0.8%	(1.86e+04, 2.16e+04)

Analysis:

- ▶ **Accuracy:** Relative error $< 2\%$ with only 50 samples
- ▶ **Efficiency:** Cost reduced from $O(n \cdot jv)$ to $O(50 \cdot jv)$
 - ▶ For Poisson $n = 900$: speedup factor $\sim 18\times$
- ▶ **Confidence:** True value in 95% interval for all tests

Practical impact: Makes trace estimation feasible for $n = 10^6$ in lattice QCD!

Experiment 4: Monte Carlo for $\det(A)$

Test Matrices: Estimate $\text{tr}(\ln(A))$, then $\det(A) = \exp(\text{tr}(\ln(A)))$

Matrix	n	Exact $\text{tr}(\ln A)$	Estimated	Rel. Error
Poisson	900	1.065e+03	1.060e+03	0.4%
VFH	625	3.677e+02	3.661e+02	0.4%
H-flow	900	5.643e+01	5.669e+01	0.4%
Pei	300	5.707e+00	5.240e+00	8.2%

Observations:

- ▶ Most matrices: excellent accuracy ($\sim 0.4\%$)
- ▶ Pei matrix (ill-conditioned, $\kappa(A) = 301$): larger error (8.2%)
 - ▶ Ill-conditioning affects logarithm sensitivity
- ▶ Heat flow matrix: only 4 Lanczos iterations needed! (symmetric structure)

Application to QCD: Determinants of Dirac operators (millions of DOFs)

$\Rightarrow$ Monte Carlo makes computation tractable

Summary and Contributions

Main Results:

1. **Unified framework** combining Lanczos, orthogonal polynomials, and Gaussian quadrature to bound $u^T f(A)v$ using only matrix-vector products
2. **Practical algorithms** for:
 - ▶ Entries of A^{-1} (diagonal and off-diagonal)
 - ▶ Trace $tr(A^{-1})$ and $tr(f(A))$
 - ▶ Determinant $\det(A)$ via $\exp(tr(\ln(A)))$
3. **Extension to non-SPD matrices** via normal equations $A^T A$
4. **Monte Carlo acceleration** reduces cost from $O(njv)$ to $O(mjv)$ with rigorous confidence bounds (Hoeffding inequality)
5. **Numerical validation** on diverse test matrices (Poisson, VFH, Wathen, etc.)

Impact: Enables trace/determinant computation for $n \sim 10^6$ in lattice QCD
⇒ Previously considered "too numerically sensitive" now tractable!

Future Work and Open Problems

Algorithmic Improvements:

- ▶ **Preconditioning:** How to accelerate convergence for ill-conditioned A ?
 - ▶ Lanczos with implicit restart or deflation techniques
- ▶ **Sharper confidence bounds:** Hoeffding inequality is conservative
 - ▶ Explore Berry-Esseen theorem or empirical bootstrap methods
- ▶ **Direct handling of $u \neq v$:** Use orthogonal polynomials with variable-sign weights instead of polarization identity

Extensions:

- ▶ **Complex non-Hermitian matrices:** Crucial for QCD (Dirac operators)
 - ▶ Bilanczos or Arnoldi-based quadrature?
- ▶ **Ultra-large scale:** $n \sim 10^9$ (next-generation lattice QCD)
 - ▶ Distributed/parallel Monte Carlo sampling